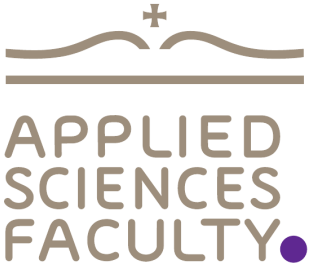
C++ – Концепції, C++20.





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Концепції

- Шаблони – дуже зручний інструмент.
- Однак, вони часто надміру загальні – конкретний код працює не для всіх можливих типів аргументів.
- Приклад: std::sort() – отримує пару ітераторів, і міг би мати такий прототип:

```
template< typename RandomIt >
void sort( RandomIt first, RandomIt last );
```

Тому вивід повідомлень про помилку для наступного коду:

```
std :: sort (1, 2);
```

буде складатися із сотень рядків незрозумілого тексту.

- Причина – тип int не вміє чогось із того, що мають вміти ітератори, тому під час інстанціації у відповідний момент, десь в глибині реалізації std::sort – в глибині коду стандартної бібліотеки, стається помилка "така операція некоректна", після чого на користувача вивалюється все – відкинуті SFINAE спроби, безуспішні перевантаження тощо.

- На аргументи шаблонів накладаються обмеження кодом, що їх використовує. Але до C++20 вони неявні і заховані в коді.
- Концепції – спосіб явно описати семантичні вимоги до аргументів шаблонів.
- Це як спрощує повідомлення про помилку, так і дозволяє здійснювати перевантаження чи різні інстанціювання в залежності від семантики.
- Приклад – замінимо прототип sort на:

```
template< std::random_access_iterator RandomIt >
void my_sort( RandomIt first, RandomIt last ){     .....     }
        ..........................
            my_sort(1, 2);
```

Повідомлення про помилку скорочується до чогось типу:

```
error: no matching function for call to 'my_sort'
note: candidate template ignored: constraints not satisfied
                                        [with RandomIt = int]
note: because 'int' does not satisfy 'random_access_iterator'
note: because 'int' does not satisfy 'bidirectional_iterator'
<skipped>
note: because '*__i' would be invalid: indirection requires
                                  pointer operand ('int' invalid)
```

- Тут використано концепцію із стандартної бібліотеки: std::random_access_iterator.

Перевантаження із допомогою концепцій

- В залежності від потужності ітераторів можуть бути потрібні різні способи сортування:

```
template< std::random_access_iterator RandomIt >
void my_sort( RandomIt first, RandomIt last ){ ..... }

template< std:: bidirectional_iterator BdIt >
void my_sort( BdIt first , BdIt last ){ ..... }

template< std::forward_iterator FwIt >
void my_sort( FwIt first , FwIt last ){ ..... }
```

- Тоді наступний код автоматично обере кращі варіанти:

```
std :: vector<int> v1;
std :: list <int> l1;
std :: forward_list <int> f1;
  .....
my_sort(v1.begin(), v1.end());
my_sort(l1.begin(), l1.end());
my_sort(f1.begin(), f1.end());
```

- Також концепції можна використати і з auto (і decltype(auto)):

```
std::random_access_iterator auto v1it = v1.begin();
//std::random_access_iterator auto v1ita = 1;
```

- Отримати зрозуміліші помилки під час компіляції можна, скориставшись static_assert() та засобами з <type_traits>, добитися такого перевантаження – з допомогою магії std::enable_if чи if constexpr, але код буде значно менш читабельним.
- Концепції мали потрапити в C++11. Ймовірно, одна із причин його затримки (це ж мав бути C++0x – де x не шістнадцяткова), але перший прототип було викинуто в 2009. Цитуючи Страуструпа: "The C++Ox concept design evolved into a monster of complexity.". Наступна спроба була під час підготовки C++17 – спрощений варіант, concepts lite[1], і нарешті в C++20 вони потрапили.

[1] Цитуючи вікі (https://en.wikipedia.org/wiki/Concepts_(C++)), "In addition to concepts themselves, "C++0x Concepts" included concept maps (a feature that could make it possible, for example, for the concept "Stack" to accept std::vector, automatically mapping "Stack" operations such as push() to differently named operations on std::vector, such as push_back()) and axioms (a facility to specify semantic properties such as associativity or commutativity, allowing the compiler to take advantage of these properties without proof)."

- Вище ми користувалися стандартними концепціями. Часто цього недостатньо – потрібно вміти формулювати свої.
- Мова надає для цього багато засобів. Почнемо із найпростішого.
- Для початку візьмемо також приклад, простіший за ітератори.
- Шаблон додавання двох чисел[2]:

```
template<typename T>
T add(const T& a, const T& b){
        return a + b;
}
```

- Питання – які числа можна додавати?
- Почнемо із floating point. Можна зробити як і раніше:

```
template<std::floating_point T>
T add2(const T& a, const T& b){
    return a + b;
}
```

Але це нецікаво.

[2]Шаблон свідомо спрощений – ніяких різних типів, std::common_type і т.д.

- У нас також є відповідний trait: std::is_floating_point_v. З його допомогою можна сконструювати концепцію так[3]:

```
template < class T >
concept my_floating_point = std::is_floating_point_v<T>;

template<my_floating_point T>
T add3(const T& a, const T& b){
    return a + b;
}
```

- Наступний крок – нам би хотілося додавати і цілі числа. Відповідна концепція (std::integral) і trait існують. Однак, перевантажувати функцію лише через це, дублюючи код, не видається адекватним. Потрібен інструмент комбінувати концепції.
- Насправді, їх існує декілька. Перший – до концепцій можна застосовувати логічні операції:

```
template < typename T >
concept my_addable_1=std::is_floating_point_v<T> || std::integral<T>;
template<my_addable_1 T>
T add4(const T& a, const T& b){
    return a + b;
}
```

[3] std::floating_point так і створений.

- Логічна комбінація концепцій – потужний інструмент, але іноді – недостатньо зручний.
- Наприклад, ми усвідомлюємо, що додавати можна не лише фундаментальні типи, але й будь які, які мають належний оператор додавання – комплексні числа, вектори тощо.
- Звичайно, можна за допомогою шаблонного метапрограмування створити відповідний trait, і з нього – концепцію. Але мова підтримує більш безпосередній спосіб:

```
template < typename T >
concept my_addable_2 = requires (T a, T b)  {
    a + b;
};
```

- Таким чином перевіряється, чи відповідний вираз компілюється.
- Як і з sizeof, decltype і т.д., ніяких обчислень виразу не відбувається – перевіряється лише, чи скомпілювався б він.
- Для таких перевірок, фактично, існує невелика мова в мові. Розглянемо її.

Способи застосувати концепції

- Нехай існує концепція:

```
template < typename T >
concept my_addable_2 = requires (T a, T b)  {  a + b;  };
```

- Використати її для аргументу шаблону можна трьома способами:

```
template<my_addable_2 T>
T add5a(const T& a, const T& b){    return a + b;   }

template<typename T> requires my_addable_2<T>
T add5b(const T& a, const T& b){    return a + b;   }

template<typename T>
T add5c(const T& a, const T& b) requires my_addable_2<T>
                                {     return a + b;   }
```

- Можна також так:

```
template<typename T>
T add5d(const T& a, const T& b) requires requires(T a, T b) {a + b;}{
    return a + b;
}
```

◇ "noexcept noexcept" тепер не єдиний – є ще "requires requires"...

- Використання із пакетами аргументів (variadic template):

```
template<typename... Args>
concept my_addable_6 = (my_addable_2<Args> and ...);

template<typename... Args> requires my_addable_6<Args...>
auto add8(Args... args) {
    return (args + ...);
}

template<my_addable_6... Args>
auto add9(Args... args) {
    return (args + ...);
}
```

- Концепції можна застосовувати до окремих методів шаблонів – оскільки вони інстанціюються лише за потреби:

```
template<typename T>
class my_vector_t{
        ..............................
  std::string to_string()
          requires std::convertible_to<T, std::string> {
                        ..........................
  }
};
```

- А ще тепер в класі може бути кілька деструкторів чи інших спеціальних методів:

```
class my_vector_t{
        ..............................
    ~my_vector_t() = default;
    ~my_vector_t() requires(!std::is_trivial_v<T>){ ..... }
}
```

- За допомогою концепцій можна створювати спеціалізації класів:

```
template <typename T>
class VectorT {  .....  };

template <typename T>
 requires  std :: invocable<T>
class VectorT<T> {  .....  };
```

- Концепції можна застосовувати до шаблонних аргументів шаблонів[4]:

```
template<
   template<typename U> requires my_addable_5<U> typename T,
   typename X>
void fn2(T<X> a, T<X> b) {}

template< template<my_addable_5> typename T, typename X>
void fn1(T<X> a, T<X> b) {}
```

[4] Clang15 має проблеми із компіляцією першого варіанту. GCC11+ і MSVC19+ – ні.

- Можна також використовувати разом з auto – як в оголошенні змінних:

```
std::random_access_iterator auto v1it = v1.begin();
```

- Так і в функціях чи лямбда-функціях:

```
std::integral auto add6(std::integral auto a, std::integral auto b) {
    return a + b;
}

auto add7 = [](std::integral auto a, std::integral auto b)
          { return a + b; };
```

- За таких використань, перший аргумент концепції береться з виведеного типу auto:

```
std::convertible_to<unsigned int> auto val{1};
```

Для std::convertible_to<T, U> тут T = int (те ж, що і decltype()), U = unsigned int.

Використання для вибору перевантажень

- Приклад вище, із перевантаженням по типу ітератора – тривіальний, розглянемо складніший. Нехай у нас є оптимізація для піднесення до цілих степенів. Тоді:

```
template<typename T>
T my_pow(T a, T b) {   .....   }

template<typename T>
T my_pow(T a, int n) {   .....   }
```

  Однак, для багатьох використань друге перевантаження буде спричиняти неоднозначність – наприклад, коли T=int.

- Виправляємо:

```
template<typename T>
T my_pow(T a, T b) {   .....   }

template<typename T> requires (!std::is_integral_v<T>)
T my_pow(T a, int n) {   .....   }
```

  Тепер працює.

◇ Ремарка: приклад чисто ілюстративний, і матиме тепер проблеми із (int, double).

Вирішення тієї ж задачі без концепцій

- SFINAE

```
template<typename T>
T my_pow2(T a, T b) {  .....  }

template<typename T>
std::enable_if_t<!std::is_integral_v<T>, T> my_pow2(T a, int n)
{   .....  }
```

- constexpr – в стилі Python:

```
auto my_pow3(auto a, auto b) -> decltype(a) {
    if ( std::is_integral_v< decltype(b)> ) {
            .....
    } else {
            .....
    }
}
```

- Хоча цей приклад надміру простий, навіть тут, видається, концепції дають більш читабельний код.
- Вибираючи перевантаження, компілятор обирає найбільш суворо обмежений концепціями варіант. Подробиці – далі.
- Кілька тематичних прикладів: `https://quuxplusone.github.io/`

Практика застосування

- Вважається, що аргументи шаблону слід (практично) завжди обмежувати концепціями – це робить код безпечнішим.

Створення концепцій

- Один спосіб розглянуто вище:

```
template < class T >
concept my_floating_point = std::is_floating_point_v<T>;
```

- Інші потребують require:

```
template < typename T >
concept my_addable_2 = requires (T a, T b)  {
    a + b; // [1]
};
```

При тому відбувається перевірка, чи вираз [1] компілюється.

- Вимог може бути декілька:

```
template < typename T >
concept my_addable_3 = requires (T a, T b)  {
    a + b;
    a+=b;
    ++a;
    a++;
};
```

- Перевірка інстанціювання – за допомогою typename:

```
template <typename T>
concept can_be_used_in_vector_1 =
                              requires { typename std::vector<T>; };
```

- Перевірка на наявність typedef:

```
template <typename T>
concept has_value_type = requires { typename T::value_type; };
```

- Перевірка на noexcept:

```
template <typename T>
concept my_addable_4 = requires (T a, T b) {
    { a + b } noexcept;
};
```

- Перевірка на можливість виклику із певним типом:

```
template <typename T>
concept can_fn_be_used_with_int = requires (T a, int b) {
    fn(a, b);
};
```

- Перевірка типу, що повертається:

```
template <typename T>
concept my_addable_5 = requires (T a, T b) {
    { a + b } noexcept -> std::same_as<T>;
};
```

◇ Ремарка: концепція std::same_as оголошена так:

```
template < class T, class U >
concept same_as = .....;
```

Тобто, параметрів два.

◇ Однак, у прикладі вище на місці першого аргументу автоматично підставляється тип результату функції ліворуч від ->.

- Тобто, цей вираз слід читати:

```
requires std :: same_as<decltype(a+b), T> == true;
```

- Той же синтаксис можна застосувати для перевірки не тільки результатів функцій:

```
requires (T v) {
  { v.size_m } -> std::convertible_to<size_t>;
}
```

- Загальне правило: перевіряючи тип, концепції передається на один аргумент менше, він автоматично береться із запиту на перевірку. Приклад із `https://en.cppreference.com/w/cpp/language/constraints`:

```
template<class T, class U>
concept Derived = std::is_base_of<U, T>::value;

template<Derived<Base> T>
void f(T); // Перевіряється Derived<T, Base>
```

- Перевірка значень булевих виразів:

```
template <typename T>
concept is_large_enough = requires (T a)  {
      requires  sizeof(T) >= 4;
};
```

◇ Самого `{ sizeof(T) >= 4; };` буде недостатньо – воно перевірятими лише, чи цей вираз скомпілюється.

- Скорочена форма:

```
template <typename T>
concept is_large_enough_2 = ( sizeof(T) >= 4 );
```

- Оголошуючи концепції, не можна використовувати (для дообмеження) інші. Такий код помилковий:

```
template<class T>
concept C1 = true;
template<C1 T>
concept Error1 = true;
```

- Явне інстанціювання та спеціалізації концепцій заборонені.
- Оголошувати концепції можна лише на рівні просторів імен.

Інші використання виразу require

- Require-вираз можна використовувати і без концепцій:

```
template <typename T>
void clever_swap(T& a, T& b) {
  constexpr bool has_member_swap = requires(T a, T b){
    a.swap(b);
  };

  if constexpr (has_member_swap) {
    a.swap(b);
  }
  else {
    using std::swap; // Допомагаємо ADL
    swap(a, b);
  }
}
```

- Приклад взято із `https://akrzemi1.wordpress.com/2020/01/29/requires-expression/`.
- Щодо допомоги ADL – див. `https://quuxplusone.github.io/blog/2022/04/30/poison-pill-redux/` – "To enable ADL, add a using-declaration".

- Можна використовувати "порожній" require[5]:

```
bool rr1 = requires{1;};
```

- Require за межами шаблонів не повертає false у випадку порушення його вимог, а спричиняє помилку компіляції.
- require може бути рекурсивним[6]:

```
requires( Iter it ) {
  *it++;
  typename Iter::value_type;
  requires requires(typename Iter::value_type v) {
    *it = v;
    v = *it;
  };
}
```

- Створення trait за допомогою require[7]:

```
template <typename Iter>
using is_iterator =
  std::bool_constant<requires( Iter it ) { *it++; }>
```

[5] MSVC19 має з цим проблеми.

[6] Приклад з: https://akrzemi1.wordpress.com/2020/01/29/requires-expression/.

[7] Приклад звідти ж.

- Створення trait за допомогою require – 2:

```
template <typename T>
using has_empty_1 =
  std::bool_constant<requires(T a) { { a.empty() } ->
                                     std::same_as<bool>; } >;
template< class T >
inline constexpr bool has_empty_v_1 = has_empty_1<T>::value;
```

- Або шаблонним метапрограмуванням C++11:

```
template<typename C>
class has_empty_2 {
    template<typename T> static constexpr auto check(T*)
    -> typename
     std::is_same<decltype(std::declval<T>().empty()), bool>::type;

    template<typename> // catch-all for types w/o empty()
    static constexpr std::false_type check (...);

    typedef decltype(check<C>(0)) type;
public:
    static constexpr bool value = type::value;
};

template< class T >
inline constexpr bool has_empty_v_2 = has_empty_2<T>::value;
```

- require перевіряє вимоги послідовно, припиняючи на першій, що не виконується. Тому некоректність наступних перевірок не зламає його компілювабельність.
- В аргументах require конструкції `(T a)`, `(T& a)`, `(T&& a)` еквівалентні.

Нюанси require-clause

- Коли за допомогою require задаються обмеження, безпосередньо дозволені лише найпростіші конструкції, що повертають bool – хоч трохи складніші потребуватимуть круглих дужок:

```
template<typename T>
    requires  std::is_floating_point_v<T>
T add10(const T& a, const T& b){
    return a + b;
}
template<typename T>
    requires  (!std::is_floating_point_v<T>) // <=== negation
T add10(const T& a, const T& b){
    return a + b;
}
```

- Достатньо прості – ім'я змінної, (виду value, value<T>, T::value тощо), ім'я концепції або require-вираз. Решта потрібно брати в круглі дужки.
- Складніші тести потребують require-виразів – розглядалося вище.

Кон'юнкція і диз'юнкція концепцій

- В require-вимозі можна комбінувати концепції (чи прості вирази) за допомогою and (&&) – кон'юнкції чи or (||) – диз'юнкції.

```
template<typename T>
 requires  std :: is_floating_point_v<T> || std:: integral <T>
T add11(const T& a, const T& b){     return a + b;     }
```

- Можна також:

```
 requires  std :: is_floating_point_v<T> or std:: integral<T>
```

- Дужки не впливають на результат – це все ще вважається двома незалежними, **атомарними**, концепціями, до яких використано диз'юнкцію – а не логічну операцію над їх результатами:

```
 requires  (std :: is_floating_point_v<T> || std:: integral <T>)
```

- Якщо хочеться таки логічних операцій, то треба робити всілякі такі фокуси:

```
 requires  (  !!( std :: is_floating_point_v<T> || std:: integral <T>))
```

Більше див. тут:
`https://akrzemi1.wordpress.com/2020/03/26/requires-clause/`.

- Розпакування пакетів аргументів не працює безпосередньо (але див. C++26 далі), але можна скористатися folding-ом:

```
template <typename... Ts>
  requires (is_trivial_v<typename Ts::value_type> || ...)
void fun(Ts... v);
```

  При цьому буде отримано одну атомарну концепцію, що перевіряє всі аргументи зразу.

- Але розбити її на атомарні можна, виділивши концепцію (не bool-вираз) для елементарної перевірки:

```
template <typename T>
concept trivial_value_type
  = std::is_trivial_v<typename T::value_type>;

template <typename... Ts>
  requires (trivial_value_type<Ts> || ...)
void fun(Ts... v);
```

- Приклади взято тут: `https://akrzemi1.wordpress.com/2020/03/26/requires-clause/`.

- C++26 додав fold expanded constraints.
- Якщо концепції розгортають той же пакет, обидві використовують той же логічний оператор і кожна елементарна концепція повністю включається (subsumes) в іншу, згортка (fold) однієї включатиме іншу.
- Завдяки цьому такий код буде працювати очікувано:

```
template <class T> concept A = std::is_move_constructible_v<T>;
template <class T> concept B = std::is_copy_constructible_v<T>;
template <class T> concept C = A<T> && B<T>;

template <class... T> requires (A<T> && ...) void g(T...);
template <class... T> requires (C<T> && ...) void g(T...);
```

Впорядкування перевантажень за допомогою концепцій

- Як вже згадувалося, обмежений концепціями шаблон має вищий пріоритет за не обмежений при виборі перевантаження.
- ◇ Завдяки цьому можна навіть так:

```
template <typename T>
class VectorT {
    .....
    ~VectorT() {     .....     }
    // Suppose we optimise here by not calling destructors
    ~VectorT() requires (std :: is_trivially_destructible_v <T>){ ..... }
};
```

- Однак, до кількох обмежених концепціями функцій застосовується поняття часткового впорядкування – partial ordering.

- Поміж кількох обмежених концепціями обирається більш обмежена – якщо компілятор може довести, що одна концепція є повністю вужчою за іншу[8]:

```
template <typename U>
concept my_standard_layout = std::is_standard_layout_v<U>;
template <typename U>
concept my_trivial_type = std:: is_trivial_v <U>;

template <typename T>
class VectorT {
            ..................
    void reserve(size_t new_size)     {   ........   }

    void reserve(size_t new_size)
     requires my_standard_layout<T>     { ........ }

    void reserve(size_t new_size)
     requires my_standard_layout<T> && my_trivial_type<T> { .... }
}
```

[8] MSVC2019 має проблеми із такими перевантаженнями.

- Скористатися `requires std::is_standard_layout_v<T> && std::is_trivial_v<T>` безпосередньо не вдасться – впорядкування концепцій спрацьовує лише для концепцій – тому їх доводиться створювати.
- А так можна було б зробити:

```
void reserve(size_t new_size)
requires my_standard_layout<T> && std::is_trivial_v<T>
{ ..... }
```

  Причина наступна.

- Для порівняння обмеженості двох функцій їх концепція спочатку відбувається нормалізація цих концепцій:
  - ◇ Повна концепція розбивається на кон'юнкції та диз'юнкції атомарних концепцій.
  - ◇ Для шаблонних концепцій `C<A1, A2, ... , AN>` відбувається підстановка їх визначення.
  - ◇ Концепції, записані простими bool-виразами вважаються атомарними самі по собі.
  - ◇ Для подальшого порівняння дві атомарні концепції вважаються однаковими, якщо вони представлені тим самим виразом, **створеним у тому ж самому місці коду**.
  - ◇◇ Через це приклад на початку не спрацює – `std::is_standard_layout_v<T>` спричиняє створення концепції наново кожен раз, коли використовується в require – і рядки коду будуть відрізнятися. Тому концепції видаватимуться ніяк не зв'язаними і між ними не буде відбуватися впорядкування.
  - ◇◇ Зауважте, таким чином C<A1, ..., AN> – concept id, синонім для оголошеної в іншому місці концепції.

- Після декомпозиції відбувається наступний аналіз:
  - ◇ C1 and C2 – більш обмежена, ніж C1.
  - ◇ C1 or C2 – менш обмежена, ніж C1.
- Для порівняння використовується булева алгебра. Подробиці – не дуже складні, але дуже громіздкі, викладено тут: `https://akrzemi1.wordpress.com/2020/05/07/ordering-by-constraints/`, в розділі "The subsumption relation".
- Ціль порівняння – довести, що одна концепція є підмножиною (частковим випадком – constraint subsumption) іншої, і вони при тому не збігаються.

Стандартні концепції C++20

- Оголошені в <concepts>, <iterator>, <ranges>, <compare>.
  - ◇ Пов'язані із ядром мови, з <concepts>: same_as, derived_from, convertible_to, common_reference_with, common_with[9], integral, signed_integral, unsigned_integral, floating_point, assignable_from, swappable, swappable_with, destructible, constructible_from, default_initializable, move_constructible, copy_constructible.
  - ◇ Концепції порівняння: boolean_testable[10] з <concepts>, equality_comparable, equality_comparable_with, totally_ordered і totally_ordered_with (з <compare>), three_way_comparable та three_way_comparable_with (з <compare>)
  - ◇ Концепції об'єктів, <concepts>: movable[11], copyable[12], semiregular[13], regular[14]
  - ◇◇ На загал, суфікс _with означає, що концепція тестує два типи, а без – один: equality_comparable<T> перевіряє чи коректним є `T{} == T{}`, equality_comparable_with<T, U>: `T{} == U{}`.

[9] Чи два типи мають спільний тип, у який можуть бути перетворений – той же, що дає std::common_type_t.

[10] Чи можна використовувати у контекстах, де потрібен bool і чи виконується звична семантика скорочених обчислень – навіть для різних типів у одному виразі.

[11] Можна переміщати чи виконувати swap.

[12] Все з movable + копіювати

[13] Все з copyable + default constructed.

[14] Semiregular + equality_comparable.

- Callable concepts[15], <concepts>: invocable[16], regular_invocable[17], predicate[18], relation, equivalence_relation[19], strict_weak_order.
- Категорії ітераторів, в <iterator>: indirectly_readable[20], indirectly_writable, weakly_incrementable, incrementable[21], input_or_output_iterator, sentinel_for[22], sized_sentinel_for[23], input_iterator, output_iterator, forward_iterator, bidirectional_iterator, random_access_iterator, contiguous_iterator[24].

[15] Див. також `https://www.cppstories.com/2021/concepts-callables/`.

[16] Об'єкт можна викликати за допомогою std::invoke.

[17] invocable + не змінює аргументів + виклики із тими самими аргументами дають однаковий результат.

[18] regular_invocable + boolean_testable результат.

[19] Див. `https://en.cppreference.com/w/cpp/concepts/equivalence_relation`

[20] Можна розіменовувати на читання.

[21] На додачу до, weakly_incrementable, можна порівнювати.

[22] "Охоронець" для input_or_output_iterator.

[23] І розмір обчислюється за константний час.

[24] random_access_iterator, і елементи розташовані послідовно в пам'яті.

- Категорії підтримки алгоритмі, в <iterator>: indirectly_unary_invocable indirectly_regular_unary_invocable, indirect_unary_predicate, indirect_binary_predicate, indirect_equivalence_relation, indirect_strict_weak_order[25], indirectly_movable, indirectly_movable_storable, indirectly_copyable, indirectly_copyable_storable, indirectly_swappable, indirectly_comparable, permutable, mergeable, sortable.
- Концепції інтервалів, з <ranges>: ranges::range, ranges::borrowed_range[26], ranges::sized_range, ranges::view, ranges::input_range, ranges::output_range, ranges::forward_range, ranges::bidirectional_range, ranges::random_access_range, ranges::contiguous_range, ranges::common_range, ranges::viewable_range.

[25] Всі indirect_ стосуються результатів розіменування.

[26] Ітератори, отримані з нього, не "провиснуть" (dangling).

- На завершення – повертаючись до сортування, з якого почали, концепція ітератора довільного доступу може виглядати так:

```
template<class I>
concept LegacyRandomAccessIterator =
    LegacyBidirectionalIterator <I>
  && std::totally_ordered<I>
  &&  requires(I  i,
   typename std::incrementable_traits<I>::difference_type  n) {
    { i  += n } -> std::same_as<I&>;
    { i  -= n } -> std::same_as<I&>;
    { i  +  n } -> std::same_as<I>;
    { n  +  i } -> std::same_as<I>;
    { i  -  n } -> std::same_as<I>;
    { i  -  i } -> std::same_as<decltype(n)>;
    {  i[n]  } -> std::convertible_to<std::iter_reference_t<I>>;
  };
```

Посилання

- Опису способів задання концепцій базується на: "C++20 Concepts: The Definitive Guide"
- Серія статей: "Requires-expression"
  - ◇ "Requires-clause"
  - ◇ "Ordering by constraints"
- "Tag dispatch versus concept overloading"
- "Concepts vs type traits".

- `https://en.cppreference.com/w/cpp/language/constraints` – загальний опис,
  - `https://en.cppreference.com/w/cpp/concepts` – загальні стандартні концепції.
  - `https://en.cppreference.com/w/cpp/iterator#C.2B.2B20_iterator_concepts` – концепції ітераторів.
  - `https://en.cppreference.com/w/cpp/iterator#Algorithm_concepts_and_utilities` – концепції алгоритмів.
  - `https://en.cppreference.com/w/cpp/ranges#Range_concepts` – концепції інтервалів.
- Концепції callable: `https://www.cppstories.com/2021/concepts-callables/`.
- Древня реалізація концепцій від boost – із часів C++98: `https://www.boost.org/doc/libs/1_79_0/libs/concept_check/concept_check.htm`.
- "if constexpr requires requires { requires } " – есей про використання різних комбінацій requires для вибору перевантажень, перевірки властивості типів тощо. Дещо іронічний.
- "Replace CRTP with concepts?"
- "Use concepts with std::remove_cvref_t", "C++26: Concept and variable-template template-parameters".

